

Project 1

Color and Image Restoration, Edge Detection, and Template Matching

Important

Pre-built libraries may be used unless otherwise stated. **Question 3 is an exception: OpenCV (cv2) may not be used for any part of the solution.** Students must implement the filtering and edge-detection procedure from scratch using basic numerical operations, building on the concepts covered in lecture. Calling a library function that performs the filtering, gradient computation, or edge selection for you does not satisfy this requirement, regardless of which package it comes from.

Questions 1–4 must be solved with classical computer vision methods only.

Submission Instructions

- **Questions 1–4:** submit a single Google Colab notebook link containing all of your code and its executed output. Present your methodology, figures, measurements, and discussion for each question in Markdown cells inside that same notebook. There is no separate writeup for Questions 1–4.
- **Question 5:** submit your predictions to the Kaggle competition, and submit one zipped folder containing your Question 5 code. Your written report and your **2-minute video explaining your approach** are uploaded as **separate files, outside the zip**. The report must follow the report template linked in the Question 5 section of this document.
- Set Colab link sharing to everyone so that we can open your notebook without requesting access.
- Declare any AI tools used in completing the project and Use of external data is not allowed.
- Do not hardcode answers or annotate results by hand. Every reported coordinate and every drawn annotation must be produced programmatically by your code.

If you use AI for any of your submissions, please attach the chain of questions so we can understand your line of thinking. Please be precise in your wording for your solutions. Remember, you are still responsible for any mistakes in the final solutions!

Project Structure

The drive links below each question contains all inputs required for Questions 1–4. Complete these questions in Python and document your results in the same Colab notebook you submit.

Point Distribution

Q1 Color Recovery	5 points
Q2 Multi-Observation Restoration	10 points
Q3 Edge Detection	10 points
Q4 Template Matching	10 points
Q5 Kaggle Competition	65 points
<i>How the 65 points of Q5 are made up:</i>	
Report and 2-minute video explaining your approach	5 points
Kaggle private leaderboard	40 points
Beating the baseline on the leaderboard	15 points
Reaching #1 at any point (one-time)	5 points

Submission Format

- **Questions 1–4:** one Google Colab notebook link. The notebook must contain all code, all generated results, and the requested figures, tables, explanations, and discussion in Markdown cells.
- **Question 5:** one Kaggle CSV submission, plus one zipped folder containing your Question 5 code. The report (following the template) and the **2-minute video** explaining your approach are uploaded **outside the zip**, as separate files.

Logistics

- **Due: Thursday, September 24, 11:59pm.**
- Everything is submitted on Brightspace: the Colab notebook link for Questions 1–4, and for Question 5 the zipped code folder plus the report and video as separate files. The Kaggle submission is made on Kaggle.
- **1 point will be deducted for every hour the project is late** (rounded down).
- There is no page limit on the Question 5 report.

General Requirements

- **Submit the notebook with all cells already executed and their outputs visible.** We grade the outputs saved in your notebook, so do not clear them before submitting. Before you submit, restart the runtime and run all cells in order, so that the saved outputs correspond to the final version of your code and the notebook runs without errors.
- Clearly display the final result for each question in the notebook, immediately below the code that produced it.
- Provide sufficient quantitative results, visualizations, and discussion to support your conclusions and design choices.
- Your submission must be self-contained. All code required to reproduce your results should be included in the submitted notebook.

1. Histogram Matching and Color Recovery

5 points

An astronomical image was recorded with an incorrect color response. The correctly rendered version of this same image is no longer available, but its red, green, and blue channel histograms were preserved. Restore the appearance of the distorted image using the supplied histogram information. The reference counts describe an image with the same dimensions as the distorted input.

Provided Files

- q1/distorted_nebula.png – the image to restore.
- q1/reference_rgb_histogram.csv – reference counts for intensities 0 through 255, with columns intensity, red, green, and blue.
- q1/reference_rgb_histogram.png – a visualization of the same reference histogram.

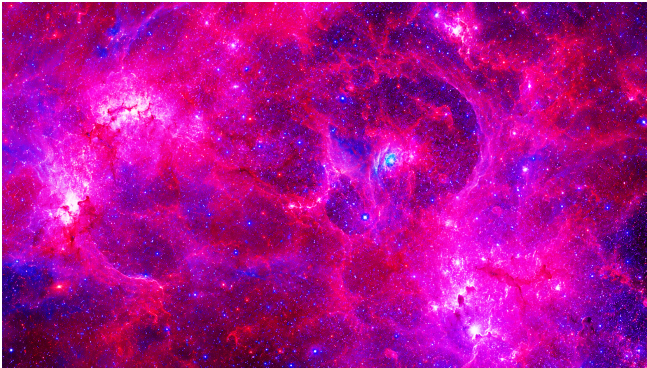


Figure 1. Distorted astronomical image

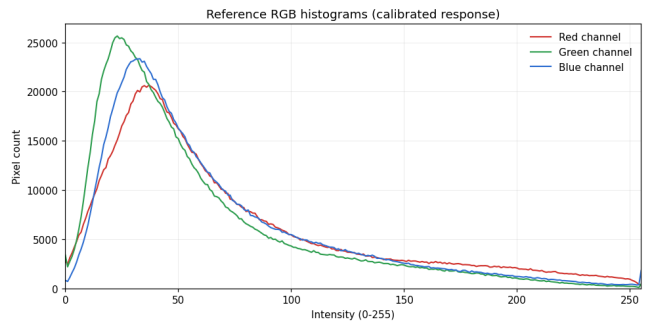


Figure 2. Reference RGB histograms

Task

Restore the distorted image as accurately as possible using the information available in the provided data. Your restoration pipeline **must include histogram matching** as one of its components. You may apply additional processing steps if they are appropriate and you clearly justify their use.

Because the original clean image is not provided, you will not be able to measure reconstruction error directly against a ground-truth image. Instead, you should evaluate the quality of your restoration using the available reference information, image statistics, visual evidence, and the behavior of the histogram before and after processing.

Your solution should demonstrate that you understand both the usefulness and the limitations of histogram matching. In particular, consider which types of image distortion can be addressed by modifying the intensity distribution and which types of information cannot be recovered through histogram matching alone.

What to Submit

- The final restored image, with the same dimensions as the distorted input image.
- The histogram of the distorted image and the histogram of the restored image, along with any additional reference histogram used in your method.
- Appropriate quantitative or visual evidence showing how the image changed as a result of your restoration procedure.
- A brief explanation of your restoration pipeline, including:
 - how histogram matching was used,
 - any additional processing steps you applied and why,
 - what aspects of the distortion histogram matching can correct, and
 - what information cannot be recovered using histogram matching alone.

Evaluation

Your restored image will be compared against the hidden ground-truth image, and your approach, evidence, and discussion will be graded alongside it. Because the ground truth is not released, design your pipeline to recover the original appearance faithfully rather than to produce a subjectively pleasing image.

Q1 - Data

2. Multi-Observation Image Restoration

10 points

Five observations show the same unknown scene. Each observation contains a different mixture of blur, noise, and intensity degradation. The useful information is distributed across the observations; no single image is guaranteed to be the best in every region. Any property you rely on when combining them, including how the observations relate to one another geometrically, is something you are expected to establish yourself and support with evidence.

Provided Files

- q2/observations/observation_1.png through observation_5.png.



Task

Develop a method that uses the five provided observations to reconstruct the clean image as accurately as possible. You may process the observations individually or combine them jointly, but your method should make meaningful use of the information available across all five images.

The clean image, its histogram, degradation labels, and region masks are not provided. You must therefore infer how to combine the observations based on their visual and statistical properties.

What to Submit

- The final (1024×810) reconstructed image.
- The reconstruction clearly displayed in the notebook.
- Appropriate visual or quantitative evidence supporting the quality of your reconstruction.
- A brief explanation of how the five observations were analyzed and combined, including the reasoning behind your chosen method.

Evaluation

Your reconstruction will be compared against the hidden clean image using PSNR and SSIM. Your grade also reflects the effective use of information from all five observations and the evidence and reasoning presented in your Markdown discussion. The clean image is not released, so support your design choices with the statistics you can measure from the observations themselves.

Q2 - Data

3. Filtering and Edge Detection

10 points

The supplied Voyager image contains curved spacecraft boundaries and thin structural elements, set against a star field, and it is affected by blur and noise. Produce an edge map that recovers the boundaries and structural elements of the spacecraft while suppressing responses caused by degradation and by the surrounding star field.

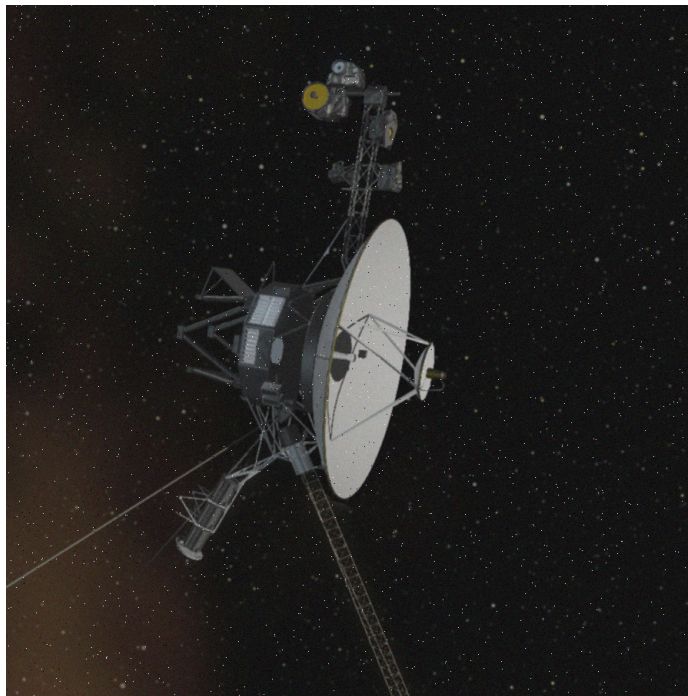


Figure 3. Degraded Voyager observation

Provided Files

- `q3/degraded_spacecraft.png` – the (900×900) image to process.

Task

Design an appropriate filtering and edge-detection procedure. You are responsible for choosing the preprocessing, edge response, and edge-selection strategy.

What to Submit

- A single (900×900) single-channel edge map whose pixels take only the values 0 and 255.
- The final edge map displayed in the notebook and a brief explanation of the method.

Evaluation

Your edge map will be compared against a reference edge map that is not released. Evaluation will consider boundary recovery, false responses, sparsity, and small localization differences.

Q3 - Data

4. Template Matching in a Large Panorama

10 points

You are given a (6000×3000) Milky Way panorama and five query images. Some queries occur in the panorama and some do not. A present query is not a pixel-identical copy of its panorama region: it has been altered in appearance, and possibly in geometry, before being given to you. Determining what kind of change you are dealing with, and designing a matching procedure that survives it, is part of the problem.



Figure 4. Milky Way search panorama

Panorama Geometry

A valid query region may cross the horizontal boundary of the panorama. In such cases, part of the matching patch may lie near the right edge while the remainder lies near the left edge. The matcher should therefore allow the panorama to be stitched horizontally, treating the left and right edges as adjacent when extracting candidate patches. No such stitching is permitted across the top and bottom boundaries.

Report the top-left corner of the matching region in panorama coordinates, with $x \in [0, 6000)$ and $y \in [0, 2820]$. The definition of the top-left corner remains unchanged when a region crosses the horizontal boundary. In such cases, (x, y) is still the top-left corner of the region in the original panorama, while the portion extending beyond the right edge is completed using the corresponding pixels from the left side of the panorama.

Provided Files

- `q4/search_scene.jpg`.
- `q4/templates/query_01.png` through `query_05.png`.

Question 4, continued: Query Matching and Localization



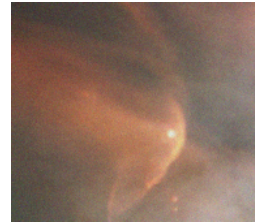
Query 1



Query 2



Query 3



Query 4



Query 5

Task

For each of the five query images, determine whether the corresponding scene is present in the provided (6000×3000) panorama. The matching region may cross the horizontal seam.

If a query is present, localize the matching (180×180) region and report the coordinates of its **top-left corner**. If a query is not present, your method must reject the match and report it as absent.

The supplied query image itself may have dimensions different from (180×180). This is because the query may have been transformed from its original source region. The coordinates you report must correspond to the original (180×180) region in the panorama, not to the dimensions of the transformed query image.

Your method should therefore make two decisions for every query:

1. Is the query present in the panorama?
2. If present, where is the matching region located?

You must also produce an annotated panorama showing every query you classify as present, as described below.

Coordinate Output

Report your results in a Markdown cell in the notebook, as a table with exactly one row per query and the columns `query_id`, `x`, and `y`:

query_id	x	y
query_01	1200	800
query_02	-1	-1

For a detected query, (x, y) denotes the **top-left corner** of the corresponding (180×180) region in the original panorama. The row above for `query_01` indicates a region whose top-left corner is located at $(1200, 800)$. For an absent query, report -1 for both x and y .

Do not include a separate presence/absence column. All coordinates must refer to the original (6000×3000) panorama. The coordinates in this table must be the ones your code computed, not values typed in by hand.

Annotated Panorama

Produce a (6000×3000) annotated version of the panorama. For every query classified as present:

- draw an axis-aligned (180×180) bounding box around the detected region, drawn as its two visible parts, one at each edge of the panorama, if the region crosses the seam, and

- label the box with the corresponding query ID.

Do not draw a bounding box for queries classified as absent.

What to Submit

- The coordinate table described above, with one row for each of the five queries.
- The annotated (6000×3000) panorama with bounding boxes and query labels.
- A brief explanation of your matching procedure and how your method determines whether a candidate match should be accepted or rejected.

Important: The location with the highest matching response should not automatically be considered a valid detection. Your method must use a justified criterion to decide whether each query is actually present in the panorama.

Q4 - Data

5. Kaggle Competition

65 points

Question 5 is the competitive component of Project 1. The Kaggle dataset, scoring metric, rules, and submission format is given on kaggle. Join the competition using the link given below.

How the 65 points are awarded

- **5 points** — your Question 5 report together with a **2-minute video** explaining your approach.
- **40 points** — your standing on the Kaggle **private leaderboard**.
- **15 points** — **beating the baseline** on the leaderboard.
- **5 points** — each student whose team reaches **#1** and beats the baseline at any point earns a one-time **+5** competition points. This includes the final private-leaderboard winner (max one time).

Your Question 5 report must follow the template below. Open it in Overleaf and use **Menu** → **Copy Project** to make your own editable copy.

Q5 - Report Template (Overleaf)

Kaggle Competition Registration and Submission

Please register your Project 1 team (solo or team of 2) in the following spreadsheet:

Project 1 Team Registration Spreadsheet

Join the Kaggle competition using the invite link below (NYU email required):

[Join here](#)

In the event of a tie on the private leaderboard, we will use the **earlier Kaggle submission time** as the tiebreaker.

You may collaborate fully with your partner. Each team submits **one** of everything: one Colab notebook link for Questions 1–4, one Kaggle CSV submission, and for Question 5 one zipped code folder, one report and one video. List both team members' names and NetIDs at the top of the notebook and in the report.

Final Submission Checklist

- The executed Python notebook contains all code and generated results.
- Every question has a Markdown cell containing its explanation, figures, and discussion.
- Colab link sharing is enabled so the notebook opens without an access request.
- The Q1 result has the same dimensions as the input.
- The Q2 reconstruction is (1024×810) pixels.
- The Q3 edge map is (900×900) pixels and contains only 0 and 255.
- The Q4 Markdown table has the columns `query_id`, `x`, `y` and contains five query rows.

- The Q4 annotated panorama is (6000×3000) pixels and contains labeled (180×180) boxes only for detected queries.
- The Q5 zip contains your code, and the report and **2-minute video** are uploaded separately, outside the zip.

Before submitting, restart the runtime, run the notebook from beginning to end, and save it with all outputs intact.

Good luck, and feel free to reach out to the TAs if you have any questions!